What Perch Proves: An Aeneas-Translated Authorization Evaluator and Its Models

Willem Wyndham and Elizabeth Engelman

The Aha Company

Abstract. This companion sets out what is proved about Perch, and about which object. Perch's authorization decision is a Rust function in the rule-core library, written in a subset that Charon and Aeneas translate into Lean. We give the definitions and all thirteen statements: the translation's equivalence with a readable model; acceptance implies permission; signer accounting; attenuation and trace inclusion; cap-window safety; delegation-tree budgets; grant containment and the containment chain; recovery invalidation in an account model; a host model of Soroban authorization; lowering equivalence; and canonical identity. Each comes with its hypotheses and its Lean names at a fixed revision. We then list the obligations on the account code around the evaluator that carry the theorems to the deployed account, the argued proposition they support, what the proofs trust, and what remains unproved.

Keywords: Formal verification · Lean · Aeneas · Authorization · Smart accounts · Soroban

1 Introduction

Perch is a Soroban smart account in which the owner installs *grants*, bounded permissions that let an app or agent act without a signature prompt for each call. The account decides each authorization with one pure function, rule-core's `rule_eval::authorize` [14], linked into the account; the contract code around it, the account *shell*, stores state, selects rules and checks signatures. The Perch paper, *Perch: Bounded Grants for Apps and Agents on Soroban Smart Accounts*[1], states five theorems about the function; this companion gives the definitions and all thirteen statements, and says what carries them to the deployed account, what they trust and what they leave unproved. Theorem and definition numbers are shared by every Perch paper.

Objects. Each statement is about one or two of five objects, none of them the compiled contract: the *model*, rule-core's hand-written Lean model of the evaluator; the *translation*, the Lean program Aeneas generates from the Rust; the *account model*, an abstract account with two events and no Rust counterpart; the *host model* of Soroban authorization, whose fidelity to soroban-env-host is

[1] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/`

tested; and Perch's *canonical-encoding package*. Aeneas gives every translated function three outcomes: success `ok`(v), failure (a Rust panic) and divergence. A theorem that a function returns `ok` on every input thus rules out panics and nontermination; a refusal is a success, Rust's `Result<_, Refusal>` being the value inside `ok`.

What "proved" means. We say *proved* only of statements Lean's kernel has checked. rule-core's CI pins all 940 exported theorems to Lean's standard axioms, `propext`, `Quot.sound`, `Classical.choice`, and checks a ledger of 180 claims: 107 proven, 40 tested, 19 assumed and 14 unproven (§11).

Reading a statement. Each statement gives its formula, its precise text, its hypotheses and its evidence: the Lean names, linked to their lines in rule-core at cbabe59 (in Perch at f0275f4 for Theorem 13), and a tag naming the object. MODEL + TRANSLATION means proved for the model and, under the `Extracted` namespace, for the translation. The HTML edition[2] adds a one-line reading and, where it helps, an instance to try. §§2–9 state the theorems by object, §10 carries them to the account, and the rest say what they trust and leave open.

2 From Rust to Lean

The evaluator, `crates/rule-eval` in rule-core, is about 2,400 lines of dependency-free `no_std` Rust that forbids `unsafe` code, in a subset Aeneas can translate: no traits, closures or iterator adaptors, and every loop a `while` over one slice.

The proof path. Charon [4] 7b462d9 reads the MIR, the compiler's mid-level representation, that rustc nightly-2026-09-17 produces, and Aeneas [5] ab34bd2 translates it into a generated 3,653-line Lean program over Aeneas's library of Rust primitives. Equivalence theorems relate it to the model through denotations (Definition 9). rule-core's CI re-extracts and fails on any difference from the committed translation, and its `lake build` rejects `sorry`, `admit`, `axiom` and similar escapes in every Lean source (rule-core `README.md:276-281`). Lean's kernel [8], v4.35.0, checks the proofs.

The shipping path. rustc 1.97.1 compiles the same source to `wasm32v1-none`, linked into the account, and nothing checks this Wasm against the translation. Tests on a natively built account bridge the paths (§10).

3 Definitions

Notation. Lists carry an arrow ($\vec{c}$); $|\cdot|$ is length or cardinality; $r_i = R_{j_i}$ is the rule chosen for context c_i; a rule's last ledger u_r is ∞ and its generation g_r is $\perp$

[2] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/verification/`

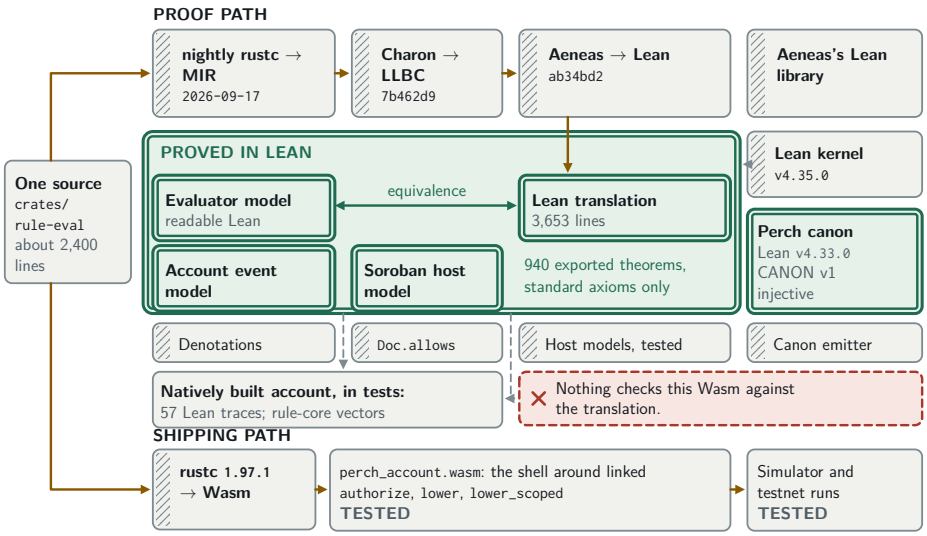


Fig. 1: One source, two paths. Lean's kernel checks theorems about the Aeneas translation of the MIR that nightly rustc produces; the account ships WebAssembly from rustc 1.97.1, and nothing checks one against the other. Double border: proved, around the objects the theorems are about. Single border: tested. Hatched: trusted, including that each model matches its subject. Dashed connectors: the tests that bridge the two paths.

when absent; $\varnothing$ is the empty records, one per rule; ledgers are natural numbers. A subscript T marks a translated function. Lean names are in rule-core at cbabe59.

Definition 1 (Contexts and authorizations)**.**

$$c \;::=\; \mathsf{call}(C, f, \vec{v}) \;\mid\; \mathsf{create}(w)$$
$$a = (\vec{c}, \vec{j}, S, \ell, g)$$

A context is one call or contract creation in an authorization entry's invocation tree. An authorization a carries the entry's contexts $\vec{c} = c_1, \ldots, c_n$ in preorder, root first; one rule index j_i per context; the claimed signers S; the ledger ℓ; and the account's grant generation g. Lean: Chain.Context; Authorization

Definition 2 (Rules and *allows*)**.**

$$\begin{aligned} \mathit{allows}(r, c, S) \iff\; & \mathit{sc}_r(c) \wedge \bigwedge_{\varphi \in \Phi_r} \varphi(c) \\ & \wedge\; k_r \le |\Sigma_r \cap S| \\ \mathit{allows}(R, c, S) \iff\; & \exists r \in R.\ \mathit{allows}(r, c, S) \\ R' \sqsubseteq R \iff\; & \forall c, S.\ \mathit{allows}(R', c, S) \\ & \implies \mathit{allows}(R, c, S) \end{aligned}$$

A rule r has a scope sc_r (one contract and a function list; false on creations), argument conditions Φ_r, signers Σ_r and a threshold k_r over distinct signers, and optionally a cap (λ_r, P_r), a last ledger u_r and a generation g_r. *allows* reads none of the last three; *authorize* checks them. A rule set allows what one of its rules allows; $R' \sqsubseteq R$ when R allows, under the same signers, everything R' allows.

Lean: Rule.allows; RuleSet.allows; RuleSet.narrower

Definition 3 (*authorize* and runs)**.**

$$\begin{aligned}
&\mathit{authorize}(R, \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \iff \\
&\quad |\vec{j}| = |\vec{c}| \wedge |\rho| = |R| \\
&\quad \wedge\ \forall i.\ j_i < |R| \wedge \ell \le u_{r_i} \wedge g_{r_i} \in \{\bot, g\} \\
&\qquad\quad \wedge\ \mathit{allows}(r_i, c_i, S) \\
&\quad \wedge\ \forall s \in S.\ \exists i.\ s \in \Sigma_{r_i} \\
&\quad \wedge\ \rho' = \mathit{charge}(\cdots \mathit{charge}(\rho, c_1, j_1) \cdots, c_n, j_n)
\end{aligned}$$

Here $r_i = R_{j_i}$. $\mathit{charge}(\rho, c, j)$ leaves ρ unchanged when R_j has no cap; otherwise it reads the amount c carries, refusing a call that carries none, and applies Definition 4's check to rule j's record. Each charge must succeed. *authorize* returns the next records or the first refusal, and a refusal changes nothing. A run applies authorizations in order, keeping the records on a refused one.

Lean: RuleSet.authorize; RuleSet.authorize_eq_ok_iff; RuleSet.run; RuleSet.Accepts

Definition 4 (Window, check and *spent*)**.**

$$\begin{aligned}
&\mathit{inWindow}_P(L, t) \iff \max(0,\, L - P) < t \le L \\
&\mathit{check}_{\lambda,P}(h, \ell, x) = h' \ \wedge\ x > 0 \implies \\
&\qquad \sum_{k:\ \mathit{inWindow}_P(\ell, t_k)} y_k \ +\ x \ \le\ \lambda \\
&\quad \wedge\ h' = ((t_k, y_k))_{k:\ \mathit{inWindow}_P(\ell, t_k)} \\
&\qquad\quad \cdot\ (\ell, x)
\end{aligned}$$

A record is a list $h = ((t_k, y_k))_{k=1}^{m}$ of positive charges in ledger order, its total their sum. The check of a charge x at ledger ℓ refuses $x < 0$; accepts $x = 0$, leaving h; and otherwise refuses a ledger before the newest charge, an overflow, 1,000 entries already kept, or a window total above λ. On acceptance it keeps the entries in the window at ℓ and appends (ℓ, x). $\mathit{spent}_j(R, \rho, \vec{a})$ lists (ℓ_a, x) for each positive charge that the accepted authorizations of run $\vec{a}$ make to rule j, in order.

Lean: Params.inWindow; Params.check; RuleSet.spent

Definition 5 (Derivation)**.**

$$\begin{aligned}
&\mathit{Derives}(R, R) \\
&\mathit{Derives}(R, R') \wedge \mathit{attenuate}(R', \sigma) = R'' \\
&\qquad \implies \mathit{Derives}(R, R'')
\end{aligned}$$

σ is one of rule-core's ten attenuation steps: drop a rule; remove functions, or restrict functions; add an argument condition; remove signers, or raise a threshold; add a cap, lower its limit, or lengthen its period; set or lower the last ledger. No step changes a rule's target or generation. $\mathit{attenuateAll}(R, \vec{\sigma}) = R'$ applies a list of steps and implies $\mathit{Derives}(R, R')$.

Lean: Chain.RuleSet.Derives; Chain.RuleSet.attenuate; Chain.Attenuation

Definition 6 (Documents and lowering)**.**

$$X = \mathit{excluded}(d_s, A)$$

For a document d on account A, $\mathit{Doc.allows}(d, A, c, S)$ holds when some rule of d covers c's contract and function, its argument constraints hold and its principals are signed by S; no rule allows a creation. $\mathit{Doc.Valid}(d)$ holds when every rule that requires all its signers names distinct signers, fewer than 2^{32} of them, and no rule has a not-after ledger of 0. $\mathit{lower}(d, A)$ is the rules of an account document, followed by the recovery rule when d configures recovery. With $X = \mathit{excluded}(d_s, A)$ for the account document d_s, which is A, the adopted controller and, if configured, the zero-knowledge adapter and pool, $\mathit{lowerScoped}(d, A, X, g)$ refuses a document that configures recovery or has a rule whose target is A, a member of X or the account's self-administration, and otherwise lowers d's rules stamped with generation g. Document conditions lower to rule-core's `ArgEq`, `ArgIn`, `ArgStrPrefix`, `ArgI128AtMost` as Table 1 says; a cap adds the upper bound of the largest `i128` on argument 2, a type guard.

Lean: Doc.allows; Doc.Valid; Doc.lower; Doc.excluded; Doc.lowerScoped

Definition 7 (Account events)**.**

$$\begin{aligned}
&\mathsf{applyDoc}(d') :\ (A, d_s, g) \mapsto (A, d', g)\\
&\mathsf{completeRecovery}(t) :\\
&\quad (A, d_s, g) \mapsto (A, t, g+1) \quad (g < 2^{64} - 1)\\
&\mathit{issue}((A, d_s, g), d)\\
&\quad = \mathit{lowerScoped}(d, A, \mathit{excluded}(d_s, A), g)
\end{aligned}$$

An account state $s = (A, d_s, g)$ is the account's address, its account document and its grant generation. $\mathsf{applyDoc}(d')$ replaces the account document; $\mathsf{completeRecovery}(t)$ installs the recovery target t and advances the generation, and is refused when g is the largest `u64`. A refused event leaves the state unchanged. $\mathit{run}(s, \vec{e})$ applies events in order, and $\mathit{completions}(s, \vec{e})$ counts the completed recoveries among them. A grant document d is *issued* in s by lowering it under the account's address, the excluded set of its account document and its generation. Lean: Account.step; Account.run; Account.completions; Account.issue

Definition 8 (Host run)**.** A model of soroban-env-host running a transaction of contract calls under enforcing authorization. Every call is a frame that a failure rolls back. `require_auth` of an address holds when the frame's direct caller is that address (invoker authority), or when an authorization entry of the address matches; on its first match the address's `__check_auth` runs on the entry's

contexts in preorder, then the expiration is checked and the nonce consumed. Stellar-account signers, `require_auth` inside `__check_auth`, `require_auth_for_args`, `authorize_as_current_contract`, contract creation as a call and source-account credentials are outside the model. Lean: Soroban.Tx.run

Definition 9 (Denotation)**.**

$$\forall x.\ \exists y.\ f_{\mathrm{T}}(x) = \mathsf{ok}(y) \wedge [\![y]\!] = f([\![x]\!])$$

$[\![\cdot]\!]$ maps the translation's values (byte vectors, `u32` and `u64` scalars, rules, caps, records, contexts, signers and outcomes) to the model's. A translated function f_{T} agrees with its model f when, for every input, it succeeds and its result denotes the model's result on the denoted input. Theorem 1 and Theorem 12 are stated this way. Lean: Equiv.denoteRules; denoteAuthorized

4 The Evaluator

Theorem 1 makes the model a faithful reading of the translation on every input; most of the other statements are also proved for the translation, and each Evidence line says which.

Theorem 1 (Total equivalence of the evaluator).

$$\begin{aligned}\forall x.\ \exists y.\ & \mathit{authorize}_{\mathrm{T}}(x) = \mathsf{ok}(y) \\ \wedge\ & [\![y]\!] = \mathit{authorize}([\![x]\!])\end{aligned}$$

Here x is the seven Rust inputs, and ok *is Aeneas's success, so neither a panic nor divergence. On every input the translated evaluator succeeds, never panicking or diverging, and its value denotes the model's result on the denoted inputs; this says nothing about allocation failure or compiled Wasm.*

Hypotheses: None: every input

Evidence: Equiv.authorize_eq · PROVED · TRANSLATION

Theorem 2 (Acceptance implies permission).

$$\begin{aligned}&\mathit{authorize}(R, \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \\ &\quad\Longrightarrow\ \forall c \in \vec{c}.\ \exists r \in R.\ \mathit{allows}(r, c, S)\end{aligned}$$

If evaluation accepts, the selected rule set allows every supplied context for the signer list it was given. The theorem takes that list as an input; the account authenticates it (obligation O1).

Hypotheses: The signer list is an input; the account authenticates it (O1)

Evidence: Chain.RuleSet.authorize_allowsAll and Extracted.authorize_allows_all · PROVED · MODEL + TRANSLATION; the chosen rule as witness: RuleSet.authorize_chosen · PROVED · MODEL

Theorem 3 (Signer accounting).

$$\begin{aligned} &\mathit{authorize}(R, \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \\ &\qquad \Longrightarrow\ \forall s \in S.\ \exists i.\ s \in \Sigma_{R_{j_i}} \\ &\mathit{signed}(r, S)\ \Longleftrightarrow\ k_r \le |\Sigma_r \cap S| \\ &S \subseteq S' \wedge \mathit{allows}(R, c, S)\ \Longrightarrow\ \mathit{allows}(R, c, S') \end{aligned}$$

Acceptance implies every supplied signer occurs in a chosen rule, and a threshold counts distinct listed identities. Adding identities preserves a rule's permission, though not whole-batch acceptance if an added identity is in no chosen rule.

Hypotheses: The signer list is an input; the account authenticates it (O1)

Evidence: Chain.RuleSet.authorize_signers; Extracted.authorize_signers; Chain.Rule.signed_iff; Chain.RuleSet.allows_of_subset · PROVED · MODEL + TRANSLATION

5 Attenuation

Attenuation derives a narrower rule set from an existing one, for example to hand part of an app's authority to a helper, by the 10 steps of Table 1 (Definition 5). Theorem 5 covers one derivation lineage: two children of one parent keep separate records, and their joint histories are not covered. Both theorems are about rule-core's model and translation; the account has no entry point that installs an attenuated grant at the pinned build, and one is proposed.

Theorem 4 (Attenuation narrows authority).

$$\mathit{Derives}(R, R')\ \Longrightarrow\ R' \sqsubseteq R$$

Successful attenuation by the ten attenuation steps yields a rule set narrower than its parent: under any signers, every call the child allows, the parent allows. Repeated derivation preserves this relationship. allows *reads no cap or expiry, so the steps that add or tighten them narrow only over runs (Theorem 5); parent and child keep separate records, and neither theorem bounds what they spend together. The account has no entry point that installs an attenuated grant at the pinned build.*

Hypotheses: Each derivation step is one of the ten attenuation steps and succeeds

Evidence: Chain.RuleSet.Derives.narrower; Extracted.Derives.narrower · PROVED · MODEL + TRANSLATION

Theorem 5 (Trace inclusion).

$$\begin{aligned} &\mathit{attenuateAll}(R, \vec{\sigma}) = R' \wedge \mathit{Accepts}(R', \varnothing, \vec{a}) \\ &\qquad \Longrightarrow\ \mathit{Accepts}(R, \varnothing, \vec{a}^{\,f}), \quad f = \mathit{chainSource}(\vec{\sigma}) \end{aligned}$$

For a single successful derivation lineage, every accepted child history from empty records is accepted by the parent from empty records, with rule choices renumbered

Table 1: The ten attenuation steps (`crates/rule-eval/src/attenuate.rs` in rule-core). Nothing else is an attenuation step. Document conditions lower to these rule-core conditions: `is-self`, `address-eq` and `u32-eq` to `ArgEq`, `string-in` to `ArgIn`, `string-prefix` to `ArgStrPrefix`. `ArgI128AtMost` appears in a lowered document only as the type check on a capped transfer's amount, with the largest `i128` as its bound, so a document cannot bound one call's amount; only an attenuation step can set a smaller maximum (rule-core `lowering.rs`).

Rule part	Steps that may narrow it	Refused
Rule set	`drop_rule` (later rule indices shift down)	none
Functions	`remove_functions`, `restrict_functions`	none
Arguments	`add_cond` (`ArgEq`, `ArgIn`, `ArgStrPrefix`, `ArgI128AtMost`)	none
Signers	`remove_signers` (threshold kept), `raise_threshold`	a lower threshold
Cap	`add_cap` to an uncapped rule, `lower_cap_limit`, `lengthen_cap_period`	a second cap, a non-positive limit, a zero period, a higher limit, a shorter period
Expiry	`lower_valid_until` (add an expiry or move it earlier)	a later expiry
Target, generation	no step	none

past dropped rules. Sibling histories are not covered. Here $\vec{a}^f$ renumbers each choice j to $f(j)$.

Hypotheses: Empty records; one derivation lineage

Evidence: Chain.RuleSet.attenuateAll_traceNarrower; Extracted.DerivesAlong.traceNarrower · PROVED · MODEL + TRANSLATION

6 Spending Limits

Theorem 6 (Cap-window safety).

$$\begin{aligned} & spent_j(R, \varnothing, \vec{a}) = ((t_k, x_k))_{k=1}^{m} \\ & \quad \Longrightarrow \ \forall L. \sum_{k:\ \max(0,\, L-P_j) < t_k \le L} x_k \ \le \ \lambda_j \\ & \quad \wedge \ \big((\forall a \in \vec{a}.\ \ell_a \ge 1) \implies \\ & \qquad \forall s. \sum_{k:\ s \le t_k < s+P_j} x_k \ \le \ \lambda_j \big) \end{aligned}$$

From empty records, in any run, counting its accepted authorizations, the charges made to a capped rule with a non-negative limit after $L - P$ and through L sum to at most its limit, for every ledger L. The model also bounds every P consecutive ledgers that start at ledger 1 or later, and those from ledger 0 when no authorization is at ledger 0.

Hypotheses: Rule j of R has the cap (λ_j, P_j) with $\lambda_j \geq 0$; empty records. The second conjunct is proved for the model only. Without its hypothesis it fails from ledger 0: two charges of the limit at ledger 0 are both recorded (`Cap.period_ledger_zero`).

Evidence: RuleSet.window_le_limit; Extracted.runAuthorizations_window_le_limit · PROVED · MODEL + TRANSLATION. RuleSet.period_le_limit_of_pos, period_le_limit · PROVED · MODEL. From ledger 0: Cap.period_ledger_zero

Ledger 0. A charge at ledger 0 lies in no window $(\max(0, L - P), L]$, so the first form of Theorem 6 holds, but each such charge is checked alone: `Cap.period_ledger_zero` records two charges of the whole limit at ledger 0, and the P ledgers from 0 then hold twice the limit. Stellar's ledgers start at 1. *On chain,* a record is bounded by one ledger entry rather than the model's 1,000 entries: at 172 bytes plus 72 per spend, the 65,536-byte limit holds 907 spends per window. Further charges are refused, which cannot break the theorem but lets a grantee exhaust its own window with small transfers.

Budgets. rule-core's delegation-tree budget, a lifetime limit, is not used by the account; it underlies a proposed agent runtime (*The Perch Vision: Recovery, Agents and the Full Account*[3]).

Theorem 7 (Delegation-tree budgets).

$$\begin{aligned} &\mathit{valid}(T) \wedge (\forall n \in T.\ \mathit{spent}_n = 0) \\ &\quad \wedge\ \mathit{run}(T, \vec{o}) = (T', \mathit{log}) \wedge T'_a = n \\ &\qquad \Longrightarrow\ \mathit{spentUnder}(T', \mathit{log}, a) \leq \mathit{allowance}_n \end{aligned}$$

$\mathit{valid}(T)$ says that every node's parent index is below its own and no allowance or spent amount is negative; run applies spends, delegations and refills in order, keeping the tree on a refused one and logging each successful spend; $\mathit{spentUnder}(T', \mathit{log}, a)$ totals the logged spends made at or under node a, and $\mathit{allowance}_n$ is that node's allowance at the end. From a valid tree with nothing spent, spending beneath every node is at most its allowance, including when its children's allowances add up to more than its own. Allowances are lifetime totals, not windows, and refills raise them.

Hypotheses: A valid tree with nothing spent

Evidence: Budget.Tree.spentUnder_le_allowance; Extracted.runBudget_spentUnder_le_allowance · PROVED · MODEL + TRANSLATION

7 Documents, Lowering and Containment

The account lowers a document into rule-core rules on install: an account document by *lower*, a grant document by *lowerScoped* under the account's address, its excluded set and the current generation (Definition 6). That Perch's compiler emits only documents satisfying `Doc.Valid` is unproven (`unproven.perch-`

[3] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/vision/`

`validation`; obligation O5). `LowerRoom` and the weaker `ArgsRoom` say the document's lists fit the evaluator's bounded vectors.

Theorem 8 (Grant target containment).

$$\begin{aligned} & \mathit{lowerScoped}(d, A, X, g) = R \\ & \quad \wedge\ \mathit{Derives}(R, R') \\ & \quad \wedge\ \mathit{allows}(R', \mathsf{call}(C, f, \vec{v}), S) \\ & \qquad \Longrightarrow\ C \neq A\ \wedge\ C \notin X \end{aligned}$$

$\mathit{lowerScoped}(d, A, X, g)$ is the rule set the account installs for grant document d at address A, with excluded set X and generation g; it allows no call to A or to an element of X, and attenuation preserves this. In the account, X is its adopted recovery controller and the zero-knowledge adapter and pool of its recovery configuration.

Hypotheses: Lowering under address A, excluded set X and generation g; for the translation, each rule's argument conditions fit the evaluator's bounded vectors (`ArgsRoom`)

Evidence: Model: Document.Doc.lowerScoped_allows_excluded, Doc.lowerScoped_attenuateAll_excluded; translation: Extracted.lower_scoped_allows_excluded, Extracted.derives_allows_excluded · PROVED · MODEL + TRANSLATION

Theorem 9 (The containment chain).

$$\begin{aligned} & \mathit{Valid}(d) \wedge \mathit{issue}(s, d) = R \\ & \quad \wedge\ \mathit{attenuateAll}(R, \vec{\sigma}) = R' \\ & \quad \wedge\ \mathit{authorize}(R', \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \\ & \qquad \Longrightarrow\ (\forall c \in \vec{c}.\ \mathit{Doc.allows}(d, A_s, c, S)) \\ & \qquad \wedge\ (\forall \mathsf{call}(C, f, \vec{v}) \in \vec{c}. \\ & \qquad\quad C \notin \mathit{excluded}(d_s, A_s)) \\ & \qquad \wedge\ (\vec{c} \neq [\,] \implies g = g_s) \end{aligned}$$

Here $s = (A_s, d_s, g_s)$ is the account state the grant is issued in, d_s its account document. A grant issued from a valid grant document d, then attenuated and accepted, allows only d's actions and no call to the account or the excluded set at issue, and a nonempty batch is evaluated at the grant's issuing generation. The account builds the excluded set from its storage (O4) and rechecks it in `__check_auth` *(*`unproven.scoped-admin-change`*).*

Hypotheses: `Doc.Valid`; for the translation, `ArgsRoom`

Evidence: Document.Account.chain; Extracted.chain · PROVED · MODEL + TRANSLATION

8 The Account Model

The account model (Definition 7) has no Rust counterpart, and only the generation matters here. The stale-rule refusal inside *authorize* is proved for the translation; the shell's part is tested (obligation O3).

Theorem 10 (Recovery invalidation).

$$\begin{aligned}
&gen(run(s, \vec{e})) \\
&\quad = gen(s) + completions(s, \vec{e}) \\
&issue(s, d) = R \ \wedge \ attenuateAll(R, \vec{\sigma}) = R' \\
&\quad \wedge \ completions(s, \vec{e}) > 0 \\
&\quad \wedge \ g = gen(run(s, \vec{e})) \\
&\quad \wedge \ authorize(R', \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \\
&\qquad \Longrightarrow \ \vec{c} = [\,]
\end{aligned}$$

In the account event model, the generation advances by exactly the number of completed recoveries, and a grant issued before one, or derived from such a grant, accepts no nonempty batch at the new generation. Administrative rules are generation-independent.

Hypotheses: The authorization carries the account's current generation (O3)

Evidence: Document.Account.run_generation; Account.issued_stale; Account.chain_after_recovery · PROVED · ACCOUNT MODEL (no Rust counterpart); stale-rule rejection: Extracted.authorize_generation · PROVED · MODEL + TRANSLATION

9 The Host Model

The evaluator's theorems take an authorization's contexts as given; Theorem 11 ties them to the host (Definition 8).

Theorem 11 (Authorization-tree coverage).

$$\begin{aligned}
&\mathsf{authorized}(A, f, \mathsf{entry}) \in new \\
&\quad \wedge \ \mathsf{checkRules}(R, S) \in program(A) \\
&\qquad \Longrightarrow \ allows(R, f, S) \\
&\mathsf{checkAuth}(A, p, \sigma, \vec{c}) \in new \\
&\qquad \Longrightarrow \ \vec{c} = preorder(p) \wedge \vec{c} \neq [\,]
\end{aligned}$$

Here new is the events a transaction's run appends. In the host model, every `require_auth` *that an account's authorization entry satisfies is for a call that every rule set the modeled* `__check_auth` *checks allows, and* `__check_auth` *receives the entry's invocation tree in preorder, root first, so never an empty batch. Authorization by invoker, as in the owner's* `execute`*, is outside the statement, and the modeled account checks the stateless allows, not Perch's shell. The model agreed with soroban-env-host 29.0.1 and 30.0.0 on every case it expresses, over 95 generated and 39 transcribed traces; Definition 8 and §12 list what it leaves out, and the simulator's host, 29.0.2, was not tested.*

Hypotheses: Contract accounts; `require_auth` satisfied by an authorization entry

Evidence: Soroban.Tx.run_authorized_allowed; Tx.run_checkAuth_contexts · PROVED · HOST MODEL (model tested against the host)

10 From the Evaluator to the Account

The account calls the Rust `lower` and `lower_scoped` on install and `rule_eval::authorize` in `__check_auth`. Theorem 12 does for lowering what Theorem 1 does for `authorize`; Theorem 13, in Perch's own Lean package, is about a grant's `doc_hash`, the SHA-256 of its CANON v1 bytes (RFC 8785 [11] restricted).

Theorem 12 (Lowering equivalence).

$$\begin{aligned}
&\mathit{LowerRoom}(d) \implies \exists R.\ \mathit{lower}_{\mathrm{T}}(d, A) = \mathsf{ok}(R) \\
&\qquad \wedge\ [\![R]\!] = \mathit{lower}([\![d]\!], [\![A]\!]) \\
&\mathit{ArgsRoom}(d) \implies \exists y. \\
&\qquad \mathit{lowerScoped}_{\mathrm{T}}(d, A, X, g) = \mathsf{ok}(y) \\
&\qquad \wedge\ [\![y]\!] = \mathit{lowerScoped}([\![d]\!], \\
&\qquad\qquad [\![A]\!], [\![X]\!], [\![g]\!])
\end{aligned}$$

Translated `lower` *returns the model's lowering when the document leaves room in its bounded vectors (*`LowerRoom`*), and translated* `lower_scoped` *does, refusals included, under the weaker* `ArgsRoom`*. For a valid document the lowered rules allow exactly what the document allows plus, when recovery is configured, every* `apply_doc` *call through the recovery rule, which the shell refuses outside a completion (tested; O8).*

Hypotheses: `LowerRoom` for `lower`, `ArgsRoom` for `lower_scoped`; `Doc.Valid` for the semantic part

Evidence: Equiv.lower_eq; lower_scoped_eq · PROVED · TRANSLATION; Extracted.lower_allows; lower_authorize · PROVED · MODEL + TRANSLATION

Theorem 13 (Canonical identity).

$$\mathit{emitDoc}(d_1) = \mathit{emitDoc}(d_2) \implies d_1 = d_2$$

Perch's canonical encoding is injective on the modeled document type; hash identity then holds up to SHA-256 collisions, an assumption outside the theorem. The theorem is about a Lean emitter written to match Perch's Rust `canon.rs` *and linked to it by golden fixtures and fuzzing, not extraction; JSON parsing and compiler correctness need separate assurance, and no theorem connects this encoding to rule-core's document type. The CANON version is not part of the hashed bytes, so a* `doc_hash` *is read under the identity the account reports (*`capabilities().doc_identity`*); three domain separations are proven:* `configPreimage_ne_emitDoc`*,* `rulePreimage_ne_configPreimage` *and* `rulePreimage_ne_emitDoc` *(rule, recovery-configuration and document preimages are pairwise distinct).*

Hypotheses: The modeled document type

Evidence: PerchFormal.Canon.emitDoc_injective; canonical_bytes_identity · PROVED · PERCH CANON PACKAGE; domain separation: configPreimage_ne_emitDoc, rulePreimage_ne_configPreimage, rulePreimage_ne_emitDoc

Table 2: Shell obligations that carry the theorems to the account. Each obligation is what the account shell must do for a statement about the evaluator to hold of the deployed account.

	Obligation	Status	Source
O1	The shell authenticates every claimed signer	tested	`gate.rs:106-113`
O2	Selection and renumbering carry the evaluator's theorems to the account	argued and tested	`gate.rs:14-16`
O3	The generation is passed in, advances only on completion, and refuses on overflow	tested; rule-core lists it as unproven against Perch `dff2a73`	rule-core `claims.yaml:2901`
O4	The excluded set matches the theorem's X, and the recheck runs	tested	`lib.rs:1225-1234`
O5	Compiler outputs satisfy `Doc.Valid`	unproven	rule-core `claims.yaml:2928`
O6	Spending records persist correctly	tested	`gate.rs:114-121`
O7	Context conversion is correct	tested	`crates/perch-rule-eval/src/convert.rs`
O8	The shell installs exactly the lowered rules, and ordinary authorization cannot select the recovery rule	tested; rule-core lists it as unproven	rule-core `claims.yaml:2940`

Perch's integration suite runs rule-core's 57 Lean-generated document traces through natively compiled `apply_doc`, `apply_scope` and `check_auth`, with the real Ed25519 verifier, enforcing authorization and the generation set from each trace, and checks the stored rules, every verdict and every spending record against the model; Perch's CI also replays rule-core's conformance vectors. The theorems hold over one fixed rule slice, while the shell builds a slice of the selected rules for each authorization; carrying them through that selection is obligation O2 (Table 2).

Proposition 1 (From the evaluator to the account; argued). *Under obligations O1–O8, Theorem 1 with* `RuleSet.authorize_chosen`*, the per-rule forms of Theorem 8 and Theorem 9 (*`Doc.lowerScoped_contract`*,* `Doc.lowerScoped_allows`*), and* `authorize_generation` *(Theorem 10) give, for each context whose selected rule belongs to a grant: the context is allowed by the grant's source document, calls neither the account nor its excluded set, and is refused after a completed recovery.*

Hypotheses: Obligations O1–O8. O1, O3, O4 and O6–O8 are tested, O2 is argued and tested, and O5, that compiler outputs satisfy `Doc.Valid`, is unproven (`unproven.perch-validation`); every grant rule satisfies `ArgsRoom` (assumed, `rule.lower-room`); the account's Wasm, built by rustc 1.97.1, computes what the translation computes (trusted; nothing checks it)

Evidence: Argued from the theorems above and the obligations; not mechanized · ARGUED

What a signature binds. A signature must commit to the rule IDs the evaluator decides over. With H = SHA-256, the account's 32-byte ID id_A, the tag τ = `perch/account/auth-addr-v1` and the selected rule IDs *ids*, an external signer

signs D:

$$P = H\big(\mathrm{XDR}(\texttt{SorobanAuthorization}\{nid, nonce, exp, inv\})\big),$$
$$B = H(\tau \,\|\, id_A \,\|\, P), \qquad D = H\big(B \,\|\, \mathrm{XDR}(\texttt{Vec}\langle\texttt{u32}\rangle\ ids)\big).$$

P is the host's payload for a legacy `Address` credential; CAP-0071's address-bound credentials [7] hash `SorobanAuthorizationWithAddress`, which adds A. A delegated signer approves the host's address-bound payload, which omits *ids*. In OpenZeppelin's account [10] at v0.8.0, an external signer signs $H(\mathrm{XDR}(\texttt{AuthDigestPreimage}\{A, P, ids\}))$ and a delegated one authorizes that preimage through `require_auth_for_args` (Table 3). The lemma is a paper proof, not mechanized.

Lemma 1 (binding). *Suppose the verifier named for an external signer correctly verifies an EUF-CMA-secure scheme and H is collision resistant. An adversary given signatures on $D(t_1), \ldots, D(t_q)$ for tuples $t_i = (A, nid, nonce, exp, inv, ids)$ of its choice outputs a $t \notin \{t_i\}$ with a signature accepted on $D(t)$ only with negligible probability.*

Proof. The map from t to D composes injective encodings with H: XDR encodes $(nid, nonce, exp, inv)$ injectively, and its union discriminant separates the two credential forms; τ is constant and id_A and P have fixed lengths; B has a fixed length and the XDR of *ids* is self-delimiting. If $D(t) = D(t_i)$ for some i, this gives a collision of H; otherwise the output is a forgery. For WebAuthn the signed message carries D as the client-data challenge. □

The lemma covers signatures on D values only; a key that also signs elsewhere, such as a Stellar account key, is covered only if no message it signs there can equal a D value. D does not depend on the contents of the rules *ids* names, the account's revision, the transaction's other entries or its fee payer.

Domain tags. Perch separates its hash domains with `perch/account/auth-addr-v1` (the payload B, scheme `addr_oz_v1`), `perch/rule` (one rule's bytes), `perch/recovery/statement`, `perch/recovery/config`, `perch/recovery/credential`, `perch/recovery/replacements` (recovery statements, configurations, credentials and replacement sets) and, as Poseidon2 constants, `perch/recovery/zk/v2/leaf`, `perch/recovery/zk/v2/bind`, `perch/recovery/zk/v2/nullifier`, `perch/recovery/zk/v2/auth`. CANON document hashes carry no tag; `configPreimage_ne_emitDoc`, `rulePreimage_ne_configPreimage` and `rulePreimage_ne_emitDoc` in Perch's canonical-encoding package prove that configuration, rule and document preimages never coincide. Each SHA-256 tag begins its preimage and none is a prefix of another, so preimages from different domains differ (by inspection, not proved). oz-policy-builder uses `prime-seat-grant` (session grants) and `prime.execution.adapter.v5` (the execution adapter's address salt).

11 Trusted Base

The proofs trust Lean's kernel (v4.35.0; v4.33.0 for the canonical-encoding package) and its axioms, `propext`, `Quot.sound`, `Classical.choice`; Charon 7b462d9, Aeneas ab34bd2 and Aeneas's Lean library, which models Rust's primitives; the MIR of rustc nightly-2026-09-17; the specification, that is the denotations and the document meaning `Doc.allows`, written from Perch's earlier account at dff2a73; the host models, tested against soroban-env-host 29.0.1 and 30.0.0; and, for Theorem 13, that its Lean emitter matches Perch's `canon.rs`, which golden fixtures and fuzzing test. **The deployed account also trusts** rustc 1.97.1 and its WebAssembly back end; the document compiler; the tested account shell, verifiers and recovery controller; and soroban-env-host.

Cryptographic assumptions. Lemma 1 assumes an EUF-CMA-secure scheme and a collision-resistant SHA-256, and Theorem 13 gives hash identity only up to SHA-256 collisions. Ed25519 [6] is checked with strict verification; P-256 is ECDSA with low S enforced; the WebAuthn verifier [13] checks neither the relying-party ID nor the origin; ML-DSA-65 is pure FIPS 204 [9] with a fixed context string. The compiler refuses a key not in its verifier's canonical form, so a threshold cannot count one key twice. The deployed Ed25519 and WebAuthn verifiers trap on a bad signature (`Error(Crypto, InvalidInput)` for Ed25519). The zero-knowledge recovery factor (*The Perch Vision: Recovery, Agents and the Full Account*[4]) proves knowledge of a secret behind a public Poseidon2 commitment, using an UltraHonk proof with KZG commitments over BN254 and Fiat–Shamir with Keccak; Perch's documents do not name the setup ceremony. The secret needs at least 128 bits of entropy, and Perch's zero-knowledge verifier is unaudited. Ed25519, P-256 and BN254 are not post-quantum, so an ML-DSA-65 signer resists a quantum adversary only if every path that can change the account's rules does. A contract ID commits to its deployer and salt, not its code, so calling fixed infrastructure code assumes that the registry deploys each pinned Wasm hash at the salt derived from it and that those contracts cannot replace their code. rule-core's claims ledger lists its 19 assumed and 14 unproven claims (Appendix A).

12 Limitations

The proofs and the binary. Nothing checks the Wasm, built by rustc 1.97.1, against the translation (Fig. 1). Release builds keep overflow checks and abort on panic, and allocation failure and stack exhaustion, which the model omits, also trap, so each fails the transaction. The specification is trusted, including `Doc.allows`, written from Perch's earlier account (dff2a73, on OpenZeppelin's library at 33726766), not from the pinned one. The owner may replace the account's code after 120,960 ledgers; the proofs cover the examined code, not its replacement.

[4] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/vision/`

The shell and the host. Tested, not proved: the JSON parser and compiler, stored-value decoding, rule-ID resolution, context conversion, the excluded set's assembly, record persistence, the credential index, the generation update, the composition of evaluation with authentication, and the recovery controller. The integration tests run the real `__check_auth` under enforcing authorization, but as natively registered contracts; the Wasm, with passkeys and real proofs, runs in Perch's `release_stack.rs` flows and on testnet. The host models are tested on selected traces and omit TTL, archival, fees, footprints and validation before the host runs. No single theorem covers the whole authorization path, including batches that mix administrative and grant rules, so Proposition 1 is argued. No external audit has been completed.

Proposed. A Lean lemma for O2, selection and renumbering, would turn the proposition's main argued step into a theorem, and a Wasm-level check (§13) would cover the shipping path. Neither has been done.

13 Related Work

Cedar [2] specifies its authorization semantics in Lean and links that model to its Rust implementation by differential random testing [3]. Here the link is a proof: Charon [4] and Aeneas [5] translate the Rust evaluator into Lean, and Theorem 1 proves the translation equal to the model on every input. The step from that Rust to the shipped Wasm stays trusted; Wasm-level tools for Soroban, Sunbeam [1] and Komet [12], could check it.

14 Conclusion

Thirteen kernel-checked statements cover Perch's decision function and lowering, through their Aeneas translation, and an account model and a host model. Proposition 1, argued, carries Theorems 1, 8, 9 and 10 to the account under eight obligations, one of them (O5) unproven; the Wasm and the specification stay trusted.

A Reproducing the Proofs

The proofs reproduce from rule-core [14] at `cbabe594ae2df408991e825e56ddd618ea29541b`, with Charon 7b462d9, Aeneas ab34bd2, Lean v4.35.0 and rustc nightly-2026-09-17, all pinned in its `flake.nix`, `rust-toolchain.toml` and `formal/lean-toolchain`. From the repository's root (rule-core `README.md:327-340`):

```
cd formal && lake exe cache get && lake build && cd ..
tools/extract.sh --check
tools/check-claims --theorems lean --skip-tests
cd conformance/generator && lake build && lake exe gen-vectors ../vectors
```

Table 3: What a signature binds. The first two rows come from the host's signature payload. OpenZeppelin columns are read from `stellar-contracts` v0.8.0 and v0.7.2 source; the 0.7.2-line build that oz-policy-builder signs for (91a2cd56…) has no recorded source, and its digest is taken from oz-policy-builder's documentation. [1]With a legacy `Address` credential; CAP-0071's address-bound credentials add the account to the host's payload. oz-policy-builder's interpreter refuses external signers in the rules it polices (error 212); read in source, not run.

Bound to	**Perch external**	**Perch delegated**	**OZ v0.8.0**	**0.7.2-line external**	**0.7.2-line delegated**
Network, nonce, signature-expiration ledger	yes	yes	yes	yes	yes
Calls in this entry's invocation tree	yes	yes	yes	yes	yes
This account	yes	yes	yes	**no**[1]	yes
Selected rule IDs	yes	**no**	yes	yes	yes
Rule contents; account revision	no	no	no	no	no
Other entries in the transaction; fee payer	no	no	no	no	no

The first line builds the model, the translation and every theorem, with its axiom record; the second fails if the committed translation differs from a fresh extraction (it needs Nix, or the `nixos/nix` image); the third checks the claims ledger; the fourth regenerates the conformance vectors and traces from the model.

Theorem 13 reproduces from Perch [15] at `f0275f41c5ab47c903bbfe9986b4bf3239c01b11`, in its `formal/` package with Lean v4.33.0 (Perch `formal/README.md:144-148`):

```
cd formal && lake build            # checks all proofs
lake env lean CheckAxioms.lean     # audits their axioms
```

The claims ledger. rule-core records 14 claims as unproven, `conformance.encoding`, `unproven.narrower-converse`, `unproven.openzeppelin-signer-mapping`, `unproven.openzeppelin-spending-limit`, `unproven.openzeppelin-authorization`, `unproven.openzeppelin-spending-limit-metering`, `unproven.openzeppelin-expiry-and-signers`, `unproven.generation-counter`, `unproven.perch-meaning`, `unproven.perch-validation`, `unproven.lowering-shell`, `unproven.scoped-admin-change`, `unproven.history-bound-on-mainnet`, `unproven.budget-shell`, and 19 as assumed, `rule.cond-room`, `rule.functions-room`, `rule.lower-room`, `rule.empty-batch`, `rule.no-creation-vocabulary`, `rule.ledgers-start-at-1`, `trusted.rustc`, `trusted.charon`, `trusted.aeneas`, `trusted.aeneas-result-loop`, `trusted.aeneas-library-items`, `trusted.lean-kernel`, `trusted.denotations`, `trusted.document-meaning`, `trusted.soroban-models`, `trusted.soroban-host`, `trusted.sdk-items`, `trusted.facade-conversion`, `trusted.host-harness`: hypotheses of the model (bounded room, the empty batch, no creation vocabulary, ledgers from 1) and the trusted items of §11.

This paper is built from shared sources with `python3 docs/perch-paper/build/paper.py` and `docs/perch-paper/latex/build.sh` in the Perch repository.

References

1. Certora: User guide for Sunbeam. Certora documentation, https://docs.certora.com/en/latest/docs/sunbeam/usage.html, contract specification and Wasm verification workflow
2. Cutler, J.W., Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Ioannidis, E., Kastner, J., Mamat, A., McAdams, D., McCutchen, M., Rungta, N., Torlak, E., Wells, A.M.: Cedar: A new language for expressive, fast, safe, and analyzable authorization. Proceedings of the ACM on Programming Languages (OOPSLA) (2024), https://www.amazon.science/publications/cedar-a-new-language-for-expressive-fast-safe-and-analyzable-authorization
3. Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Kastner, J., Mamat, A., McCutchen, M., Rungta, N., Shah, B., Torlak, E., Wells, A.: How we built Cedar: A verification-guided approach. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion) (2024), https://arxiv.org/abs/2407.01688
4. Ho, S., Boisseau, G., Franceschino, L., Prak, Y., Fromherz, A., Protzenko, J.: Charon: An analysis framework for Rust. arXiv:2410.18042 (2024), https://arxiv.org/abs/2410.18042
5. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. In: Proceedings of the ACM on Programming Languages (ICFP) (2022), https://arxiv.org/abs/2206.07185
6. Josefsson, S., Liusvaara, I.: Edwards-curve digital signature algorithm (EdDSA). RFC 8032 (2017), https://www.rfc-editor.org/rfc/rfc8032
7. Kozhevin, D.: CAP-0071: Authentication delegation and address-bound Soroban credentials. Stellar Core Advancement Proposals (2025), https://github.com/stellar/stellar-protocol/blob/master/core/cap-0071.md, final; protocol 27
8. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Automated Deduction – CADE 28, LNCS 12699. pp. 625–635 (2021), https://doi.org/10.1007/978-3-030-79876-5_37
9. National Institute of Standards and Technology: Module-lattice-based digital signature standard. FIPS 204 (2024), https://csrc.nist.gov/pubs/fips/204/final
10. OpenZeppelin: Stellar smart account contracts, v0.8.0 (ddc327e). GitHub, OpenZeppelin/stellar-contracts, packages/accounts (2026), https://github.com/OpenZeppelin/stellar-contracts/tree/ddc327eb92d63709436da3a8455f641c2d5476c5/packages/accounts/src, context rules, signers and policies
11. Rundgren, A., Jordan, B., Erdtman, S.: JSON canonicalization scheme (JCS). RFC 8785 (2020), https://www.rfc-editor.org/rfc/rfc8785
12. Runtime Verification: Komet, revision a078e29. GitHub, runtimeverification/komet, https://github.com/runtimeverification/komet/tree/a078e296621a4f52fd1497fed64435e9948cc830, pinned contract-semantics source
13. W3C: Web authentication: An api for accessing public key credentials, level 2. W3C Recommendation (2021), https://www.w3.org/TR/webauthn-2/
14. Wyndham, W.: soroban-rule-core, revision cbabe59. GitHub, bitswell/soroban-rule-core (2026), https://github.com/bitswell/soroban-rule-core/tree/cbabe594ae2df408991e825e56ddd618ea29541b, Rust evaluator, model, translation and proof trust inventory
15. Wyndham, W., Engelman, E.: Perch source, revision f0275f4. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry__perch/tree/f0275f41c5ab47c903bbfe9986b4bf3239c01b11, account, compiler, recovery, verifiers and canonical-document proof